

課題「Web Design Conference」

このファイルの目的

- 先生の添削時にコードの意図を伝えやすくするため
- 自分自身の振り返り・備忘録として
- マークダウン記法の練習として

課題URL

- GitHub
<https://github.com/Narui-Tomoka/-WDC->
- Figmaデザイン
<https://www.figma.com/design/KCeV5O1Gmf7UKLhi6y4ViO/wdc?node-id=1-19&t=IWYyx3J9OKC0bYLs-1>

使用技術

言語・ライブラリ

- HTML
- CSS (scss)
- JavaScript

開発環境・ツール

- Vite
- Autoprefixer (VS Code 拡張機能)
- markdownlint (VS Code 拡張機能／Markdown プレビュー用)

活用した自動計算サイト

- aspect-ratio 自動計算
https://aspect.arc-one.jp/#google_vignette
- clamp() 自動計算
<https://min-max-calculator.9elements.com/?16>

今回の課題で心がけたこと

メンテナンス性の考慮

前回の課題では SCSS でネストを多用して特異性が上がり、メンテナンスしにくいコードになってしまった。

その反省を活かし、可能な限りネストは **3階層程度まで** に抑え、メンテナンス性を意識した。

ヘルパークラスの活用

今回の課題では BEM の命名規則を利用したが、BEM のみだと汎用性の低いクラスが増えてしまうため、汎用性のあるヘルパークラスも作成した。

クラス名をただで内容が想像できることを意識して命名した。

例

- フォント指定用 : `f-mont`
- フォントウェイト指定用 : `bold`
- 幅制限用 : `inner`

Figma デザインを反映しやすい CSS 設計

html に `font-size: 62.5%`、body に `font-size: 1.6rem` を指定。

これにより **1rem = 10px 相当** となり、px ベースの Figma デザインからの変換がしやすくなった。

※実際のブラウザ上では body の指定により本文サイズは 16px になる。

レスポンシブ対応を意識し font-size 等に clamp() を使用

メディアクエリを書かなくても画面サイズに応じて font-size 等が自動調整されるよう、`clamp()` で最小値・推奨値・最大値を指定した。

今回の課題では明示的なレスポンシブ指定はなかったが、画面幅を狭めてもある程度崩れにくいレイアウトにできた。

2026/2/3 追記 : ブレイクポイント

ブレイクポイントについては以下の数値に設定した。

- TAB 1024px以下
- SP 767px以下

2026/2/3 追記 : レスポンシブ対応について

2025年で使われているスマホの約90%が画面幅375px以上であることから、画面幅375pxまでレイアウトが崩れないようにレスポンシブ対応を行った。

画面幅が変化してもコンテンツが画面端につかない工夫

各セクションに以下を指定 :

```
padding-inline: max(16px, calc((100% - 1100px) / 2));
```

これにより、

- 画面が広いとき → 中央寄せ

- 画面が狭いとき → 最低 16px の余白確保が実現できる。

CSS 変数（CSS カスタムプロパティ）の利用

前回の課題では font-size に `clamp()` を多用し、管理が煩雑になった。

そこで今回は再利用しそうな値を `:root` に CSS 変数として定義し、値の共通化とメンテナンス性向上を図った。

入力しやすいフォーム

input 要素に `name` 属性や `autocomplete` 属性を付与し、値を適切に設定（name, email, tel など）することで、ブラウザの自動補完機能が働くようにした。

ユーザーの入力負担を減らすフォーム設計を意識した。

備忘録

:root に書く CSS 変数（カスタムプロパティ）

:root に変数を書く目的

```
:root { --main-color: #ff7a00; }
```

- JS から触れる
- テーマ切り替えができる
- CSS としてそのままブラウザに残る

Sass の\$変数との使い分け

Sass の\$変数はビルド時に消えて**ただの値になる**ので

- 設計・計算・レイアウト用 → Sass 変数\$
- 色・余白・テーマ → CSS 変数--

具体的な使い方

1. _root.scss ファイルを作成し以下の例のような内容を書く。

```
:root {  
  
  /_ color _/  
  --color-main: #ff7a00;  
  --color-text: #222;  
  --color-bg: #fff;  
  
  /_ font _/  
  
  --font-base: "Noto Sans JP", sans-serif;  
  --fs-base: clamp(1.4rem, 1.2rem + 0.5vw, 1.6rem);
```

```
/_ spacing _/
```

```
--space-xs: 0.4rem;  
--space-sm: 0.8rem;  
--space-md: 1.6rem;  
--space-lg: 3.2rem;  
}
```

2. main.scss で読み込む(@use)

3. CSS 変数を使う

```
.button {  
  
  background-color: var(--color-main);  
  color: var(--color-bg);  
  padding: var(--space-sm) var(--space-md);  
  font-family: var(--font-base);  
}
```

`_index.scss` の使い方

Sass における `_index.scss` は、フォルダ内ファイルをまとめ、外部から読み込みやすくするためのファイル。

基本的な役割

フォルダに `_index.scss` を置くと、他ファイルからフォルダ名だけで読み込める。

基本の書き方 (@forward)

現在の Sass では `@import` ではなく `@forward` を使うのが標準。

フォルダ構成例

```
src/scss/  
├── global/  
│   ├── _variables.scss  
│   ├── _mixins.scss  
│   └── _index.scss <-- ① ここで変数を転送  
└── components/  
    └── _button.scss <-- ② ここで読み込んで使う
```

① global/_index.scss (変数の集約・公開) @forward を使って、外部から global フォルダを読み込んだ時に変数を使えるようにする。

```
// global/_index.scss  
@forward 'variables';  
@forward 'mixins';
```

② components/_button.scss (変数の利用)

変数を実際に使うファイルでは、@use を使って読み込む。このとき、パスは _index.scss があるディレクトリを指定するだけで済む。

```
// components/_button.scss
@use '../global'; // index.scss を自動的に読み込む

.button {
  // 「ディレクトリ名.変数名」でアクセスする
  background-color: global.$primary-color;
  // 名前空間 (global) を省略して使いたい場合は 'as _' を指定
  // @use '../global' as _;
  // color: $text-color;
}
```

書き方のポイント

1. 名前空間の活用

@use '../global' とすると、global.\$変数名 の形で使える。
エイリアスを付けることも可能。

2. 同じファイルを二度読みしても問題ない

@use や @forward は重複出力されない。

3. プライベート変数の保護

変数名の先頭に - または _ を付けると、@forward しても外部から参照できない。

clamp()の値の指定

- 自動計算ツールに入力する最大ビューポートは**コンテンツ幅**が基準 (inner等幅制限があればその値)
- フォントサイズの最小値は、デザイン時の指定がなければ参考サイトなども基準に考える。
参考サイト: <https://www.gpol.co.jp/blog/292/>

推奨されるHTML構造

一般的なWeb制作のセオリーに当てはめると、以下のような構成がベスト。

1. header (ヘッダー)

一番上の「WDC」というロゴや、右上の「ABOUT / SPEAKER / SESSION」といったナビゲーションメニューが含まれる部分。

範囲: 画面最上部の細い帯の部分。

2. main (メイン)

サイトの「メインコンテンツ」すべてを包む。

各セクションは、すべてこの main の中に入る。

範囲: FV (WORK REVOLUTION) から Register (お申し込みフォーム) まで。

3. footer (フッター)

一番下の「©2030 Web Design Conference」と書かれたコピーライトの部分。

headerが他のsectionの上に重なる場合

今回のheaderは見た目上はFVに重なって見えても、header と main は兄弟要素として分けるのがよい。

理由：HTMLの構造（セマンティクス）は「役割」で決めるものだから。

1. HTML構成 header は main の外に出したままにする。
2. CSSでの重ね方 header に position: absolute; を使い、main（または fv）の上に浮かせて配置。

このような構造が推奨される理由

- HTMLのルールを守れる:
header は「サイトの案内役」なので、独立している方が検索エンジン（SEO）に優しくなる。
- 管理がしやすい:
後から「やっぱりヘッダーだけ全ページ共通にしたい」となった時に、FVの中から引っ張り出す手間がない。
- アクセシビリティ:
スクリーンリーダーなどは、まず header を探し、次に main を探す。この順番が崩れないのが理想的。

プルダウンメニューの作り方

HTMLでプルダウンメニュー（ドロップダウンリスト）を作成する最も基本的な方法は、<select>要素と<option>要素を使用すること。

1. 基本的なプルダウンメニュー

```
<label for="fruits">好きな果物を選んでください</label>
<select name="fruits" id="fruits">
  <option value="apple">りんご</option>
  <option value="banana">バナナ</option>
  <option value="orange">オレンジ</option>
</select>
```

- ### 2. よく使われる属性 selected: 最初から特定の項目を選択状態にする。 disabled: その項目を選択不可にする。 required: フォーム送信時に選択を必須にする。

```
<select name="pref" required>
  <option value="">都道府県を選択してください</option>
  <!-- プレースホルダーとして利用 -->
  <option value="tokyo" selected>東京都</option>
  <option value="osaka" disabled>大阪府（準備中）</option>
</select>
```

3. グループ化 (<optgroup>)

項目をカテゴリーごとに分類したい場合に使用する。

```
<select name="cars">
  <optgroup label="国産車">
    <option value="toyota">トヨタ</option>
```

```
<option value="honda">ホンダ</option>
</optgroup>
<optgroup label="外車">
<option value="bmw">BMW</option>
<option value="mercedes">メルセデス</option>
</optgroup>
</select>
```

4. 応用：クリックで開くメニュー（CSS/JS）

フォームではなく、ナビゲーションメニューとしてデザイン性を高めたい場合は、タグとCSS（ホバーやクリックイベント）を組み合わせで作成するのが一般的。